

Synchronization Tools in Operating Systems

Prof. Mehdi Pirahandeh

1 Introduction

In this lecture, we will cover the critical concepts of process synchronization, which is essential for managing shared resources and ensuring data consistency when multiple processes are running concurrently. Topics include the critical-section problem, race conditions, various synchronization tools like semaphores and mutexes, and hardware support for synchronization.

2 Background

In a system with multiple processes or threads, managing access to shared resources (memory, files, etc.) becomes crucial. Without proper synchronization, race conditions can occur, leading to incorrect or unpredictable outcomes.

2.1 Race Condition Example

A race condition occurs when two or more processes attempt to update shared data at the same time as shown in Figure 1. For instance, two processes, P0 and P1, might both attempt to update a variable 'next_available_pid'. Without synchronization, the same 'pid' could be assigned to two different processes, leading to inconsistent states.

3 The Critical-Section Problem

The critical-section problem involves ensuring that only one process executes a critical section at any time. The solution to this problem must satisfy three requirements:

- **Mutual Exclusion:** Only one process can execute in the critical section at a time.

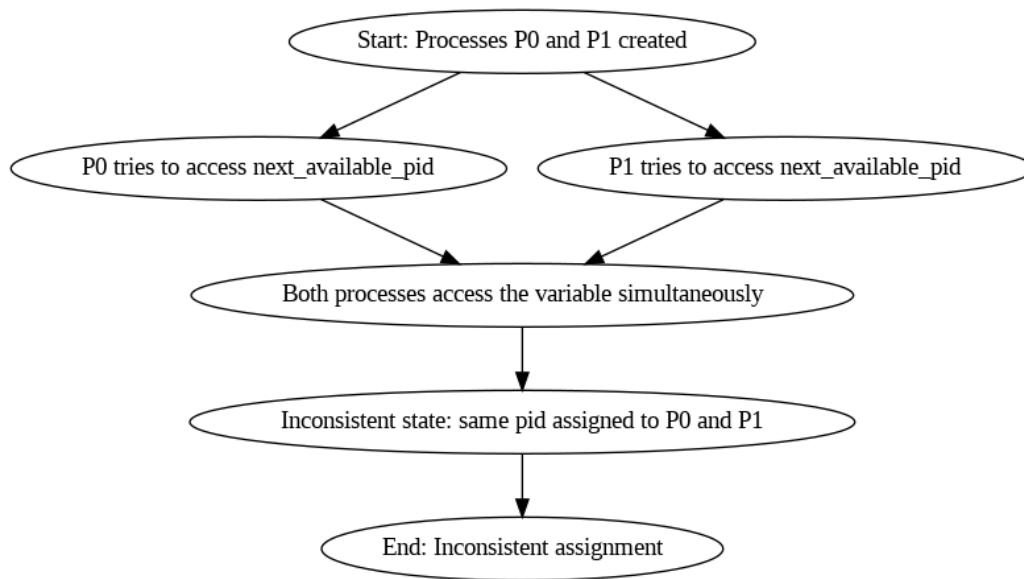


Figure 1: race condition flowchart

- **Progress:** If no process is in the critical section, the decision of which process will enter next must be made without indefinite delay.
- **Bounded Waiting:** A limit must exist on the number of times other processes are allowed to enter the critical section after a process has requested to enter.

As shown in Figure 2, the flowchart explains the critical-section problem and the necessary conditions that must be met to ensure the correct execution of processes that require access to the critical section. Below is a step-by-step breakdown of the flowchart:

3.0.1 Start

The process begins with multiple processes (denoted as $P_0, P_1, \dots, P_{n-1}$). These processes may require access to the same shared resource or data, which is managed within the **Critical Section**. The objective is to ensure that only one process can execute in the critical section at a time to avoid conflicts or inconsistencies.

3.0.2 Request

Each process must first **request** permission to enter the critical section. This request is a part of the protocol that ensures the process will not disrupt other

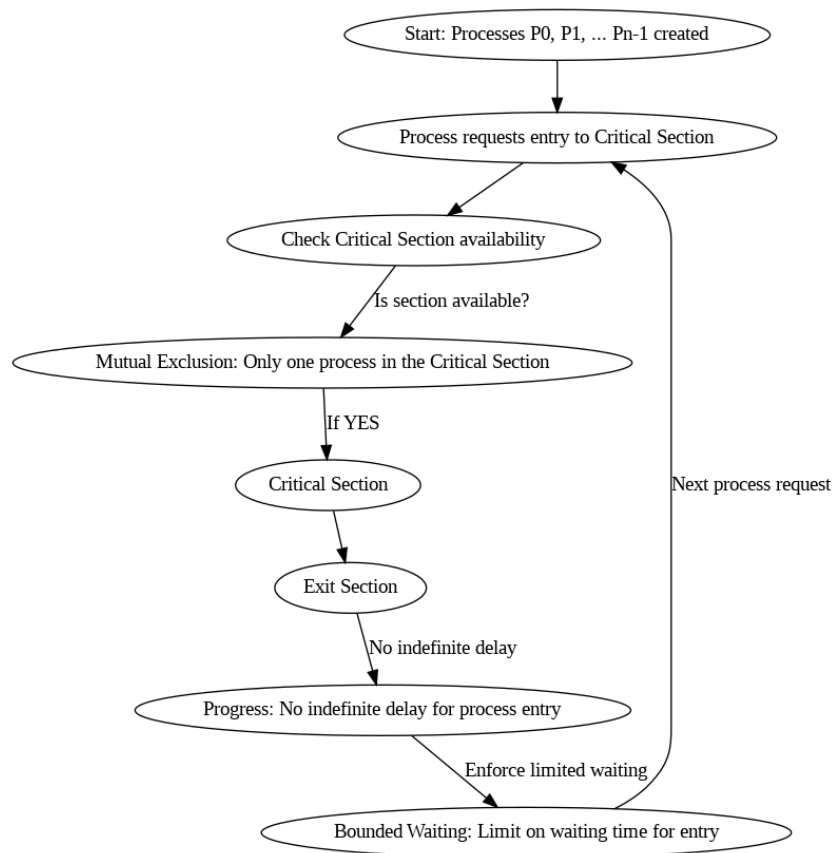


Figure 2: The Critical-Section Problems Flowchart

processes already in the critical section.

3.0.3 Check

Once a request is made, the system checks whether the critical section is available. This is a crucial point in the flowchart where the decision is made about whether the requesting process can enter the critical section or needs to wait.

3.0.4 Mutual Exclusion

If the critical section is available (i.e., no other process is currently inside), the requesting process is granted entry, thereby enforcing **Mutual Exclusion**. This condition ensures that **only one process is allowed in the critical section at any given time**, preventing simultaneous access to shared resources.

3.0.5 Critical Section

Once mutual exclusion is guaranteed, the process enters the **Critical Section**. This is where the process performs its task, such as updating shared variables or accessing shared resources. During this period, no other process is allowed to enter the critical section.

3.0.6 Exit Section

After completing the operations in the critical section, the process exits. Upon exit, two important requirements must be satisfied to ensure the correct functioning of the system: **Progress** and **Bounded Waiting**.

3.0.7 Progress

The **Progress** condition ensures that if no process is currently in the critical section, the decision on which process will enter next will not be postponed indefinitely. This prevents a scenario where processes are unnecessarily delayed, ensuring the system continues to move forward.

3.0.8 Bounded Waiting

The **Bounded Waiting** condition ensures that there is a bound (limit) on how long a process must wait before it is allowed to enter the critical section. This prevents a situation known as *starvation*, where a process might wait

indefinitely due to other processes continuously gaining access to the critical section.

3.0.9 Re-entry and Next Process

Once the bounded waiting condition is met, the next process may request access to the critical section. The cycle then continues, ensuring that each process eventually gains access to the shared resource while satisfying mutual exclusion, progress, and bounded waiting. The flowchart clearly illustrates how a system must handle multiple processes requiring access to the critical section, while ensuring three essential conditions: **Mutual Exclusion**, **Progress**, and **Bounded Waiting**. These conditions are necessary to prevent race conditions, process starvation, and to maintain overall system consistency and efficiency.

4 Solutions to the Critical-Section Problem

Various methods can be employed to solve the critical-section problem, including hardware and software solutions.

4.1 Peterson's Solution

Peterson's algorithm is a two-process solution to the critical-section problem. It uses two shared variables, 'turn' and 'flag[2]', to ensure mutual exclusion and progress:

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
    /* Critical section */
    flag[i] = false;
    /* Remainder section */
}
```

This algorithm ensures that both mutual exclusion and bounded waiting are satisfied. However, it is not guaranteed to work on modern architectures due to instruction reordering.

4.2 Memory Barrier

On modern architectures, memory barriers are used to prevent the reordering of instructions, ensuring that memory operations are completed before others begin. For instance:

```
while (!flag)
    memory_barrier();
print x;
```

5 Hardware Support for Synchronization

Some systems provide hardware instructions to support synchronization, such as ‘test_and_set’ and ‘compare_and_swap’. These instructions ensure that critical section code can be executed atomically.

5.1 Test-and-Set Instruction

This instruction tests and modifies the content of a memory location atomically:

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}
```

This function is used in algorithms like spinlocks, where the lock is acquired by continually testing the variable until it is available.

5.2 Compare-and-Swap Instruction

The compare-and-swap instruction swaps the values of two variables only if the original value matches a given condition:

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

6 Mutex Locks and Semaphores

To simplify the handling of the critical section problem, OS designers have introduced mutex locks and semaphores.

6.1 Mutex Locks

Mutexes provide a simpler way to manage the critical section. A mutex ensures mutual exclusion by allowing only one process to acquire the lock at a time.

6.2 Semaphores

Semaphores are more sophisticated synchronization tools that support both counting and binary semaphores. The two operations used with semaphores are ‘wait()’ and ‘signal()’:

```
wait(S) {  
    while (S <= 0);  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

Semaphores can be used to solve various synchronization problems, including ensuring that one process waits for another to complete a task.

7 Monitors

Monitors are high-level synchronization constructs that allow multiple processes to access shared data, but only one process can be active in the monitor at a time.

7.1 Monitor Example

```
monitor ResourceAllocator {  
    boolean busy;  
    condition x;  
    void acquire(int time) {
```

```

        if (busy)
            x.wait(time);
        busy = true;
    }
    void release() {
        busy = false;
        x.signal();
    }
}

```

8 Liveness Issues

Liveness refers to the set of properties ensuring that processes make progress. Common issues include:

- **Deadlock:** Two or more processes wait indefinitely for an event that can only be triggered by one of the waiting processes.
- **Starvation:** A process may never get the chance to execute if others continuously preempt it.
- **Priority Inversion:** A higher-priority process waits for a lower-priority one to release a lock.

9 Comparison of Synchronization Methods

The following table compares different synchronization methods:

10 Examples and Applications

10.1 Using Mutex for File Access

Suppose two processes need to write to the same file. By using a mutex lock, we can ensure that only one process writes to the file at a time:

```

mutex lock;
lock.acquire();
/* Critical section: write to file */
lock.release();

```

Method	Type	Advantages	Disadvantages	Use Case
Peterson's Solution	Software	Simple, ensures mutual exclusion	Fails on modern architectures	Educational purposes
Test-and-Set	Hardware	Fast, atomic operation	May cause busy waiting	Spinlocks
Compare-and-Swap	Hardware	Atomic, avoids busy waiting	Complex to implement	Lock-free algorithms
Mutex Locks	Software	Simple to use, no busy waiting	Limited to mutual exclusion	General critical section
Semaphores	Software	Solves complex synchronization	Incorrect use can cause issues	Process co-ordination
Monitors	Software	High-level abstraction	Only available in certain languages	Structured synchronization

Table 1: Comparison of Synchronization Methods

10.2 Using Semaphores for Process Synchronization

Two processes, P1 and P2, need to ensure that one process executes before the other. A semaphore initialized to 0 can solve this:

```
P1: wait(sem); /* P1 waits for P2 */  
P2: signal(sem); /* P2 signals P1 */
```

11 Conclusion

This lecture has explored the fundamental tools and techniques for process synchronization in operating systems. Understanding these tools is crucial for designing systems that can handle concurrent processes without issues like race conditions, deadlocks, or starvation. These methods, ranging from low-level hardware instructions to high-level abstractions like monitors, provide essential mechanisms for ensuring the correctness and liveness of processes in modern operating systems.